Document number: P0766R1
Audience: EWG

Ville Voutilainen
2017-10-15

Fixing small-ish functionality gaps in constraints

Abstract

This paper deals with some small-ish fallout parts of the merge of Concepts into the C++20 working draft; namely, requires-clauses in lambdas, auto as a function parameter type, and requires-clauses in template template parameters.

requires-clauses in lambdas

Currently, lambdas cannot have requires-clauses. One of the arguments for that lack I've heard mentioned is that lambdas are supposed to be short, and requires-clauses make lambdas verbose.

The problem with that argument is that it focuses on the wrong thing; it's, at least to me, much more important that lambdas can be

- local - in general, lambdas allow me to write function objects without having to write namespace-scope definitions
- refactored from functions, or from functors, or from functor templates

I think it very likely to have function templates that use plain requires-clauses. If I then want to refactor such a function template into a polymorphic lambda, I'm now going to face a massive hindrance, I have to turn the requires-clauses into "proper" Concepts first. I will need to introduce namespace-scope (Concept) definitions where none were required before the refactoring.

I don't think that's just a consistency issue. I envision writing implementation-detail function templates with requires-clauses rather than proper Concepts when such a Concept is not all that reusable. I will choose to not write a Concept because it's not going to be a cross-cutting Domain Concept, it's just a simple compile-time contract of a function. For such uses, the scope of that contract should be allowed to be as narrow as feasible, and it should not be forced to be any wider than the scope of the function parameter declaration.

auto as a function parameter type

We happily allow auto as a lambda parameter type, in which case the lambda is polymorphic. We currently don't allow a plain function to have auto as a parameter type.

This, again, hurts refactoring. There's also no ambiguity about whether such a function is a template, it's obviously a template, its parameter type is a keyword that suggests the type can be anything, and that the type is deduced.

I'm perfectly willing to admit that this is not a real functionality gap. I can write an unconstrained template instead of a function with an auto parameter. The problem there is that none of the arguments against a terse syntax for a constrained function template hold, and now I need to do tedious mechanical transformations when doing a simple refactoring from a polymorphic lambda to a function template. Sure, for lambdas that capture

something, there is a fair amount of additional mechanical transformation involved, but for non-capturing lambdas there shouldn't be.

requires-clauses in template template parameters

Curiously, our specification doesn't allow requires-clauses in template template parameters, but the gcc implementation of the Concepts TS does.

Again, in order to constrain a template template parameter without having to write a namespace-scope concept definition, we need to support a requires-clause here. Not every constraint should be a concept.

requires-clauses as a migration step

From practical adoption experience, I deem it very likely that the first step in moving from existing constraints to concepts is to turn enable_ifs into requires-clauses. That's to me a very good reason to not leave gaps in our support for requires-clauses.

There's a perhaps more controversial and a much scarier aspect to it. Some codebases will need to try and make pre-C++20 and C++20 constraints look the same on the declaration level. This will mean that macros get employed, but those macros can't work if there's no syntactic position that works for an enable_if and a requires-clause. The pre-C++20 constraints can't use the syntactic position for a constrained-template-parameter, but they can use the syntactic position for a requires-clause.

requires-clauses are necessary, not just a simple rewrite

There are use cases where using a requires-clause is necessary; the other syntaxes cannot do what a requires-clause can. A very prominent example is establishing relations between elements of multiple packs. Such relationships can currently be established with functions but not with lambdas. While a function can be called with multiple 'raw' packs and a lambda can't (the first pack must be provided, the subsequent pack must be deduced), it's still possible to have multiple wrapped packs:

```
[](tuple<Args1...>, tuple<Args2...>) requires (PackRelation<Args1, Args2> &&...)
```